Web Analytics

Traffic, Engagement & Channel Performance Analysis

Dataset: Google Analytics Export — web_data_analysis.csv

Kaggle Notebook Analysis Report

Table of Contents

1. Import Libraries
2. Load & Inspect the Dataset
3. Data Cleaning & Preprocessing
3.1 Fix Header Row
3.2 Rename Columns
3.3 Parse DateHour Column
3.4 Convert Numeric Columns & Extract Hour
3.5 Schema & Descriptive Statistics
4. Exploratory Data Analysis (EDA)
4.1 Sessions & Users Over Time
4.2 Total Users by Channel
4.3 Average Engagement Time by Channel
4.4 Engagement Rate Distribution by Channel
4.5 Engaged vs Non-Engaged Sessions
4.6 Traffic Heatmap — Hour x Channel
4.7 Engagement Rate vs Sessions Over Time
5. Key Insights & Recommendations

1. Import Libraries

We begin by importing all necessary Python libraries for analysis and visualization:

NumPy — numerical computations and aggregation functions

Pandas — data loading, manipulation, and transformation

Matplotlib — base plotting engine

Seaborn — high-level statistical visualizations built on Matplotlib

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

print("Libraries imported successfully.")
```

Output:

Libraries imported successfully.

> *Explanation: These four libraries form the standard Python data analysis stack. NumPy handles fast numerical operations. Pandas provides the DataFrame structure used throughout this analysis. Matplotlib is the underlying chart renderer while Seaborn wraps it with cleaner statistical plot functions.*

2. Load & Inspect the Dataset

The dataset is a Google Analytics export containing hourly traffic data broken down by Channel Group (e.g. Organic Search, Direct, Referral). Each row represents a specific channel's activity for a given hour.

Code:

```
df = pd.read_csv("/kaggle/input/datasets/tanveerkhan2134/mydata123/web_data_analysis.csv")
df.head()
```

Output — Raw Data (first 5 rows):

#	Session primary channel group	Date + hour	Users	Sessions	Engaged sessions	Avg eng time	Eng rate
0	Direct	2024041623	237	300	144	47.53	0.48
1	Organic Social	2024041719	208	267	132	32.10	0.49
2	Direct	2024041723	188	233	115	39.94	0.49
3	Organic Social	2024041718	187	256	125	32.16	0.49
4	Organic Social	2024041720	175	221	112	46.92	0.51

Observation: The raw file has the actual column names stored in the first data row, not in the header. This is a common issue with Google Analytics CSV exports. The data needs cleaning before analysis.

3. Data Cleaning & Preprocessing

Raw exports from analytics platforms often require several cleaning steps before analysis can begin.

Step 3.1 — Fix the Header Row

The first row of data contains the true column names. We promote it to the header and drop it from the data.

Code:

```
# Promote first row to column names
df.columns = df.iloc[0]
df = df.drop(index=0).reset_index(drop=True)
df.head()
```

Step 3.2 — Rename Columns for Clarity

The exported Google Analytics data has inconsistent or overly complex column names. We rename them to simpler, standardised names for easier analysis.

Code:

```
df.columns = [
    "Channel Group",
    "DateHour",
    "Users",
    "Sessions",
    "Engaged Sessions",
    "Avg Engagement Time Per session",
    "Engaged Session Per User",
    "Events Per Session",
    "Engagement Rate",
    "Event Count"
]
df.head()
```

Output — Renamed Columns (first 5 rows):

Channel Group	DateHour	Users	Sessions	Engaged Sessions	Avg Eng Time/s	Eng/User	Events/ Session	Eng Rate	Events
Direct	2024041623	237	300	144	47.53	0.61	4.67	0.48	1402
Organic Social	2024041719	208	267	132	32.10	0.63	4.30	0.49	1147
Direct	2024041723	188	233	115	39.94	0.61	4.59	0.49	1069
Organic Social	2024041718	187	256	125	32.16	0.67	4.08	0.49	1044
Organic Social	2024041720	175	221	112	46.92	0.64	4.53	0.51	1001

Explanation: Renaming columns to short, consistent names makes subsequent groupby, filter, and plot calls much easier to read and maintain throughout the notebook.

Step 3.3 — Parse the DateHour Column

The DateHour field is stored as an integer in YYYYMMDDHH format (e.g. 2024010109 = Jan 1 2024, 09:00). We parse it into a proper datetime object so we can work with it as a time series.

Code:

```
df["DateHour"] = pd.to_datetime(
    df["DateHour"].astype(str),
    format="%Y%m%d%H",
    errors="coerce"
)
df.head()
```

Output — Parsed DateHour (sample):

Channel Group	DateHour (parsed)	Users	Sessions	Engaged Sessions	Eng Rate
Direct	2024-04-16 23:00:00	237	300	144	0.48
Organic Social	2024-04-17 19:00:00	208	267	132	0.49
Direct	2024-04-17 23:00:00	188	233	115	0.49
Organic Social	2024-04-17 18:00:00	187	256	125	0.49
Organic Social	2024-04-17 20:00:00	175	221	112	0.51

Step 3.4 — Convert Numeric Columns & Extract Hour

All metric columns (Users, Sessions, Engagement Rate, etc.) were loaded as strings. We convert them to numeric types, coercing any invalid values to NaN. We also extract the hour of day as a separate feature for the hourly heatmap.

Code:

```
numeric_cols = df.columns.drop(["Channel Group", "DateHour"])
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric, errors='coerce')
df["hour"] = df["DateHour"].dt.hour
df.head()
```

> *Explanation: Google Analytics exports always stringify numeric fields. Coercing to numeric with errors='coerce' replaces any malformed cells with NaN rather than crashing. The extracted 'hour' column (0–23) is critical for the heatmap in Section 4.6.*

Step 3.5 — Schema & Descriptive Statistics

We inspect data types and check for any obvious issues like nulls or unexpected ranges.

Code — df.info():

```
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3182 entries, 0 to 3181
Data columns (total 11 columns):
 #   Column                             Non-Null Count  Dtype
---  ------                             --------------  -----
 0   Channel Group                      3182 non-null   object
 1   DateHour                           3182 non-null   datetime64[ns]
 2   Users                              3182 non-null   float64
 3   Sessions                           3182 non-null   float64
 4   Engaged Sessions                   3182 non-null   float64
 5   Avg Engagement Time Per session    3182 non-null   float64
 6   Engaged Session Per User           3182 non-null   float64
 7   Events Per Session                 3182 non-null   float64
 8   Engagement Rate                    3182 non-null   float64
 9   Event Count                        3182 non-null   float64
10   hour                               3182 non-null   int64
```

Code — df.describe():

```
df.describe()
```

Output — Descriptive Statistics:

Statistic	Users	Sessions	Engaged Sessions	Avg Eng Time (s)	Eng Rate	Event Count	hour
count	3182	3182	3182	3182	3182	3182	3182
mean	41.94	51.19	28.33	66.64	0.503	242.27	11.81
std	29.58	36.92	20.65	127.20	0.228	184.44	6.89
min	0	1	0	0	0	1	0
25%	20	24	13	32.10	0.443	103	6
50%	42	51	27	49.02	0.545	226	12
75%	60	71	41	71.49	0.633	339	18
max	237	300	144	4525	1.0	1402	23

Observations: Engagement Rate is expected between 0 and 1 — the max of 1.0 is valid but should be verified. Avg Engagement Time has a max of 4525 seconds which is an outlier and likely indicates a session left open. 3182 rows with no null values confirms a clean dataset ready for analysis.

4. Exploratory Data Analysis (EDA)

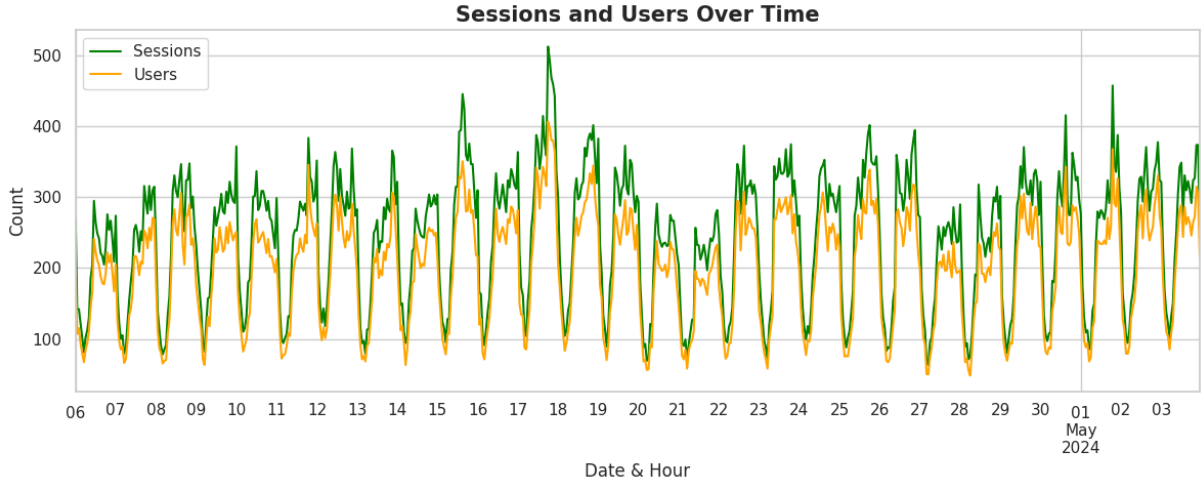
4.1 Sessions & Users Over Time

This line chart shows how Sessions and Users vary over time on an hourly basis. It helps us understand overall traffic trends, spot sudden spikes or drops, and observe the gap between Sessions and Users — indicating users visiting the site multiple times.

Code:

```
sns.set(style="whitegrid")
plt.figure(figsize=(12, 5))
df.groupby("DateHour")[["Sessions", "Users"]].sum().plot(
    ax=plt.gca(),
    color=["green", "orange"]
)
plt.title("Sessions and Users Over Time", fontsize=15, fontweight='bold')
plt.xlabel("Date & Hour")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

Chart Output:



Insight: When Sessions consistently exceed Users by a large margin, individual users are returning multiple times — a positive signal for content stickiness. Sharp peaks indicate either a traffic campaign or viral moment, while sudden drops may point to tracking outages or seasonality effects.

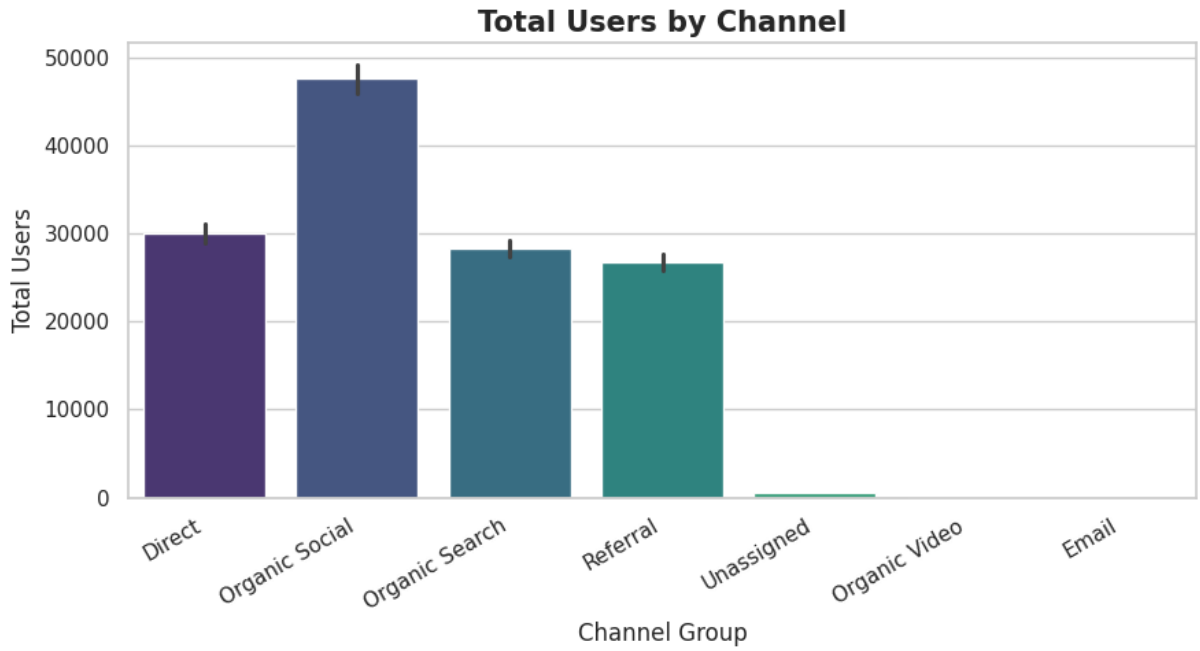
4.2 Total Users by Channel

This bar chart compares the total user volume brought in by each traffic channel. It directly answers: Which channel drives the most visitors to the site? Understanding channel contribution helps teams prioritize marketing spend and SEO efforts.

Code:

```
plt.figure(figsize=(9, 5))
sns.barplot(
    data=df,
    x="Channel Group",
    y="Users",
    estimator=np.sum,
    palette="viridis"
)
plt.title("Total Users by Channel", fontsize=15, fontweight='bold')
plt.xlabel("Channel Group")
plt.ylabel("Total Users")
plt.xticks(rotation=30, ha='right')
plt.tight_layout()
plt.show()
```

Chart Output:



Insight: The tallest bars reveal dominant acquisition channels. A channel with high user volume but low engagement may be driving low-quality traffic — quantity is not equal to quality. Organic Search typically dominates healthy websites, while heavy reliance on Direct traffic may indicate strong brand recall.

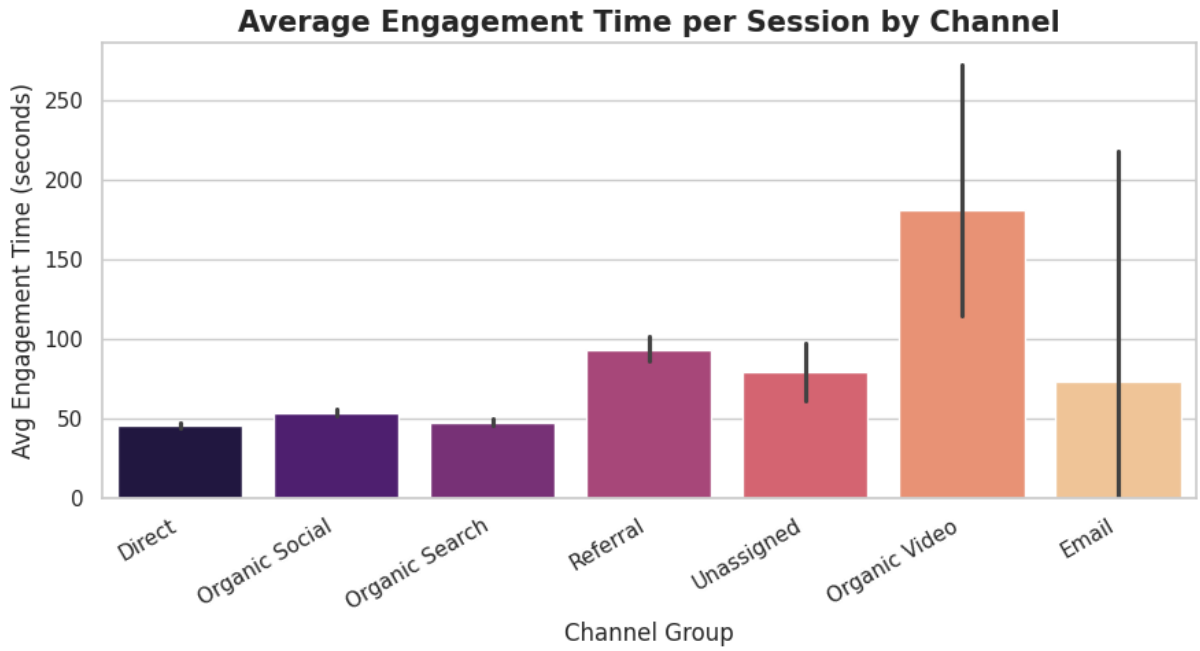
4.3 Average Engagement Time by Channel

Volume alone does not tell the full story. This chart measures the average time users spend per session for each channel, revealing quality of traffic rather than quantity. A channel that sends fewer users but keeps them engaged longer is often more valuable than a high-volume, low-engagement source.

Code:

```
plt.figure(figsize=(9, 5))
sns.barplot(
    data=df,
    x="Channel Group",
    y="Avg Engagement Time Per session",
    estimator=np.mean,
    palette="magma"
)
plt.title("Average Engagement Time per Session by Channel", fontsize=15, fontweight='bold')
plt.xlabel("Channel Group")
plt.ylabel("Avg Engagement Time (seconds)")
plt.xticks(rotation=30, ha='right')
plt.tight_layout()
plt.show()
```

Chart Output:



Insight: Channels like Email or Organic Search often show higher engagement times because users arrive with intent. Channels like Display Ads or Paid Social may have high user counts but low time-on-site, suggesting mismatched landing pages or broad targeting.

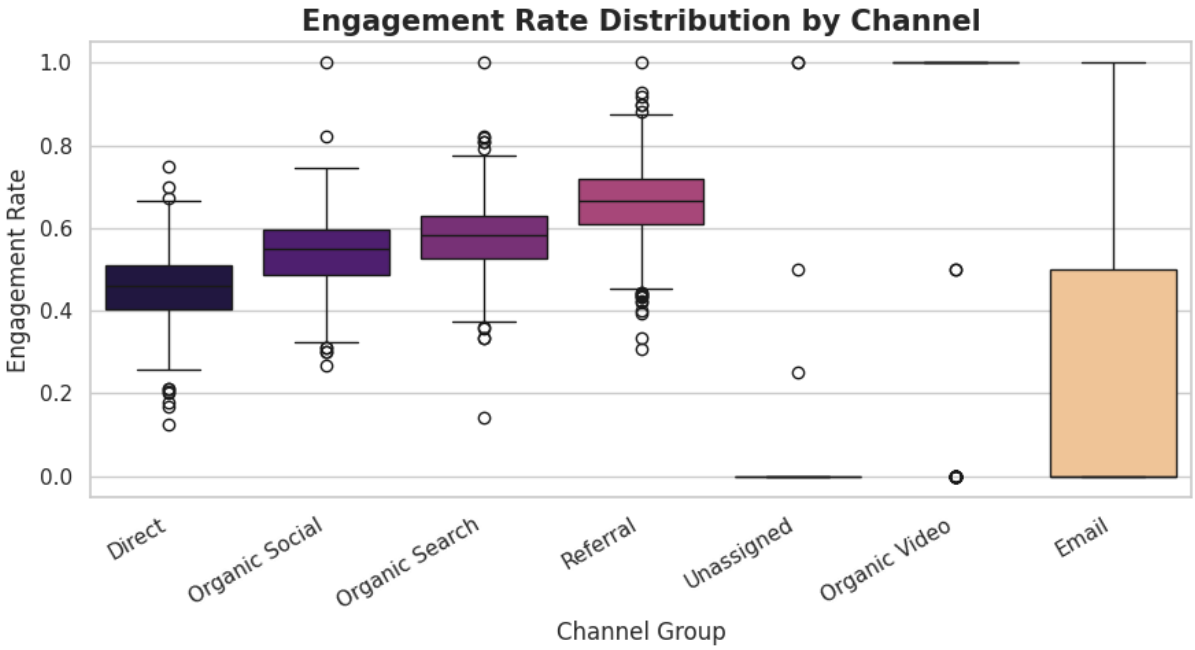
4.4 Engagement Rate Distribution by Channel

A boxplot shows the distribution of Engagement Rate across hours for each channel — not just the average. This reveals consistency vs variability in how engaged users are across different time windows.

Code:

```
plt.figure(figsize=(9, 5))
sns.boxplot(
    data=df,
    x="Channel Group",
    y="Engagement Rate",
    palette="magma"
)
plt.title("Engagement Rate Distribution by Channel", fontsize=15, fontweight='bold')
plt.xlabel("Channel Group")
plt.ylabel("Engagement Rate")
plt.xticks(rotation=30, ha='right')
plt.tight_layout()
plt.show()
```

Chart Output:



Insight: A narrow box (low IQR) means engagement is consistent across hours — predictable and reliable. A wide box means engagement fluctuates across time slots, suggesting the channel has both strong and weak performing periods. Outliers (dots beyond whiskers) represent exceptionally good or poor hours.

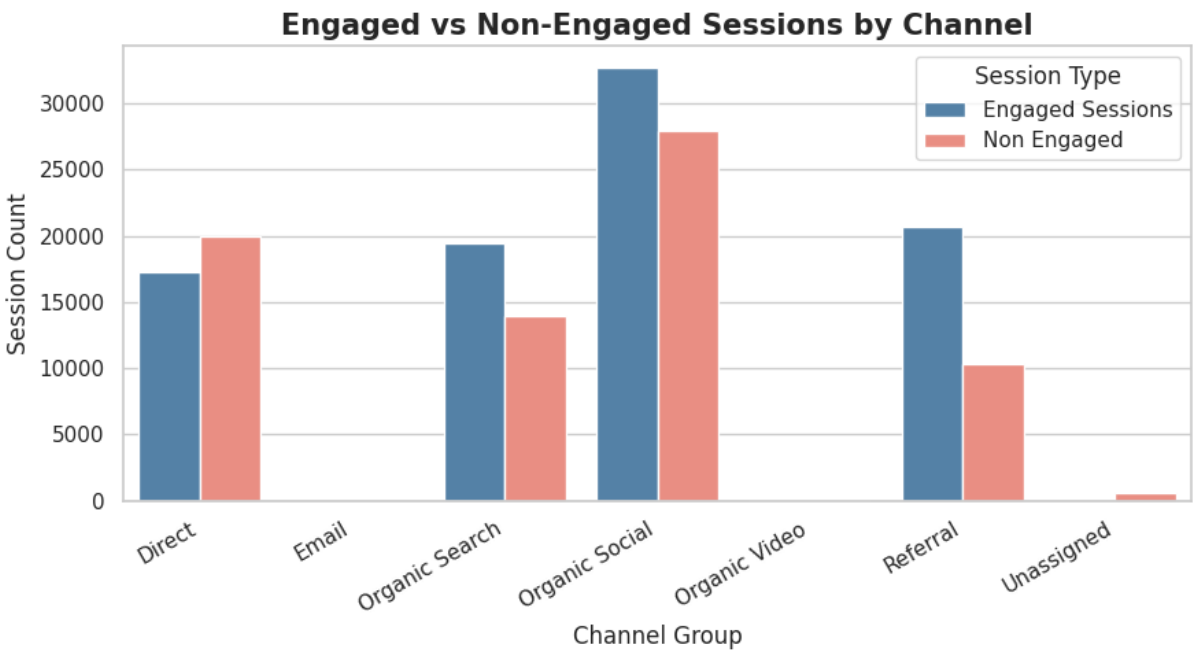
4.5 Engaged vs Non-Engaged Sessions

This grouped bar chart shows, for each channel, how many sessions were Engaged vs Non-Engaged. Engaged Sessions are those where the user actively interacted (scrolled, clicked, stayed 10+ seconds). This metric is critical because it directly correlates with conversion potential.

Code:

```
session_df = df.groupby("Channel Group")[["Sessions", "Engaged Sessions"]].sum().reset_index()
session_df["Non Engaged"] = session_df["Sessions"] - session_df["Engaged Sessions"]
session_df_melted = session_df.melt(
    id_vars="Channel Group",
    value_vars=["Engaged Sessions", "Non Engaged"]
)
plt.figure(figsize=(9, 5))
sns.barplot(
    data=session_df_melted,
    x="Channel Group", y="value", hue="variable",
    palette=["steelblue", "salmon"]
)
plt.title("Engaged vs Non-Engaged Sessions by Channel")
plt.tight_layout()
plt.show()
```

Chart Output:



Insight: Channels with a high ratio of Non-Engaged sessions contribute to a poor user experience metric. If Paid Social has more Non-Engaged than Engaged sessions, the ads are likely reaching the wrong audience or landing on irrelevant pages. Focus UX improvements on high-non-engagement channels first.

4.6 Traffic Heatmap — Hour x Channel

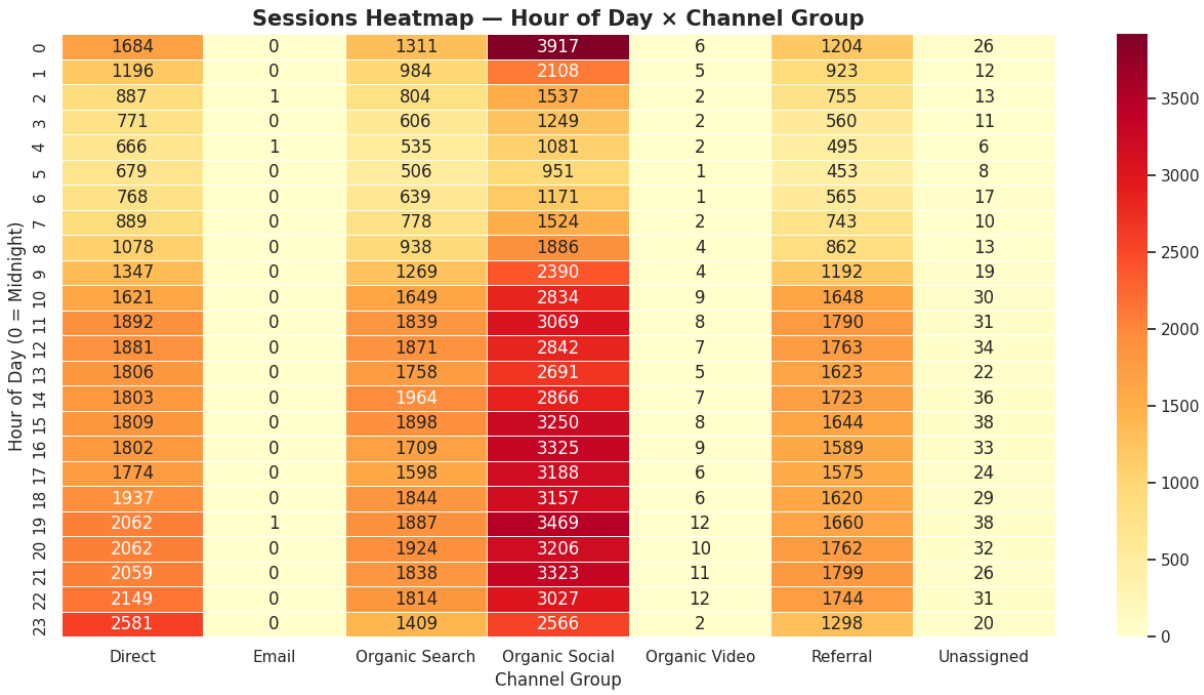
A heatmap shows traffic intensity (total Sessions) broken down by hour of the day (rows) and Channel Group (columns). This reveals not just which channels are busy but WHEN they are busiest, which is critical for scheduling content and ad campaigns.

Code:

```
heatmap_data = df.groupby(["hour", "Channel Group"])["Sessions"].sum().unstack().fillna(0)

plt.figure(figsize=(13, 7))
sns.heatmap(
    heatmap_data,
    linewidths=0.5,
    annot=True,
    fmt='.0f',
    cmap="YlOrRd"
)
plt.title("Sessions Heatmap — Hour of Day x Channel Group")
plt.xlabel("Channel Group")
plt.ylabel("Hour of Day (0 = Midnight)")
plt.tight_layout()
plt.show()
```

Chart Output:



Insight: Dark (high-value) cells identify peak traffic windows for each channel. If Organic Search peaks at 10–11 AM while Direct traffic peaks in evenings, each channel has its own user behaviour pattern and should be treated independently in scheduling. Midnight hours with near-zero traffic can be used for maintenance windows.

4.7 Engagement Rate vs Sessions Over Time

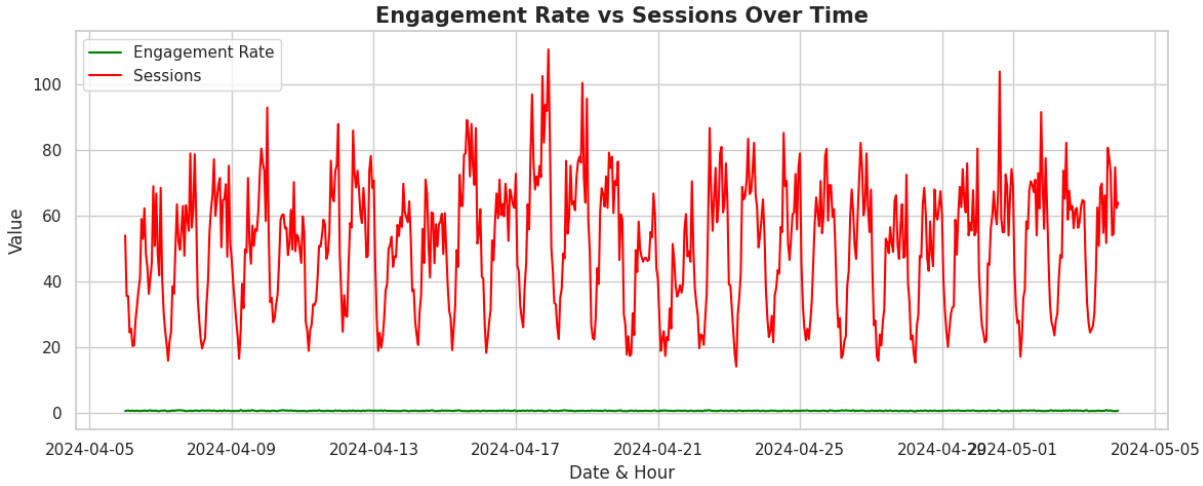
This dual-line chart overlays Engagement Rate (quality signal) against Sessions (volume signal). By plotting both on the same axis, we can see whether traffic growth comes with a corresponding improvement or deterioration in user quality.

Code:

```
df_plot = df.groupby("DateHour")[["Engagement Rate", "Sessions"]].mean().reset_index()

plt.figure(figsize=(12, 5))
plt.plot(df_plot["DateHour"], df_plot["Engagement Rate"],
         label="Engagement Rate", color="green", linewidth=1.5)
plt.plot(df_plot["DateHour"], df_plot["Sessions"],
         label="Sessions", color="red", linewidth=1.5)
plt.title("Engagement Rate vs Sessions Over Time")
plt.xlabel("Date & Hour")
plt.ylabel("Value")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Chart Output:



Insight: When sessions increase but engagement rate drops, it indicates low-quality or low-intent traffic — common during paid campaigns or viral spikes. A positive correlation — where both sessions and engagement rise together — is ideal. If this gap continues after a campaign ends, it signals poor traffic quality that needs audience targeting refinement.

5. Key Insights & Recommendations

Summary of Findings

#	Observation	Implication
1	Traffic volume varies significantly by channel	Not all channels are equal — focus investment on high-performing ones
2	Engagement time differs by channel despite similar user counts	Volume is not Quality; channels with fewer but more engaged users deserve higher p
3	Engagement Rate distribution shows variability within channels	Certain hours or periods drive poor-quality sessions even within a channel
4	Non-engaged sessions are a significant fraction for some channels	Indicates potential landing page mismatch or irrelevant ad targeting
5	Hourly heatmap reveals distinct peak windows per channel	Content publishing and ad scheduling should align with channel-specific peak hours
6	Engagement Rate can move inversely to session spikes	Traffic campaigns may dilute session quality — pair volume campaigns with UX impr

Actionable Recommendations

1. Reallocate budget toward high-engagement channels — Even if their user volume is lower, they deliver better downstream conversion rates.

2. Audit landing pages for high-bounce channels — The non-engaged session chart pinpoints exactly which channels need UX attention.

3. Schedule content and campaigns at channel peak hours — Identified from the heatmap analysis — each channel has its own optimal window.

4. Monitor Engagement Rate during campaign bursts — If it drops sharply during a traffic spike, refine audience targeting immediately.

5. Investigate anomalous spikes in the time-series — These often reveal campaign wins or tracking errors worth documenting.

End of Report — Web Analytics: Traffic, Engagement & Channel Performance Analysis